1 Memory Units and MIPS

1.1 Memory Units

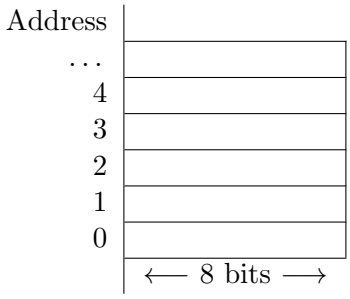
A binary digit (a bit) is a single 0/1 digit. A byte (nowadays) is a unit of 8 bytes, and a kilobyte (KB) is $2^{10} = 1024$ bytes, a megabyte $2^{20} = 1024$ KB, then GB, TB, PB, EB, and so on.

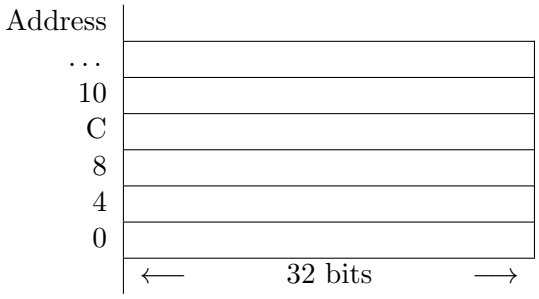
1.2 Working unit size – Words

A word is a natural unit of data used by a given processor. In this course MIPS32 is used, but MIPS16 and MIPS64 do exist. This typically indicates the sizes of registers, memory addresses, instruction sizes, and floating point variable sizes.

1.3 Memory Organisation

We have an "array" of bytes, or words.





1.4 Big and Little Endian

Consider the word `0xBABACAFE`. To store it in address 8, we have two options: Big Endian has most significant byte at the lowest memory address, and Little Endian the least significant. Also illustrated below is the 32-bit integer 9 stored in memory address 4:

Offset / Address	0	1	2	3
...				
10				
C				
8	BA	BA	CA	FE
4	**0**	**0**	**0**	**9**
0				

Big Endian

Offset / Address	0	1	2	3
...				
10				
C				
8	FE	CA	BA	BA
4	**9**	**0**	**0**	**0**
0				

Little Endian

It turns out that it doesn't really matter which Endian is used, but most modern computers use Little Endian.

1.5 MIPS Registers

The MIPS32 CPU has 32 registers: $\$0, \ldots, \$31$, each of size 32 bits. Different registers serve different purposes; for example, \$0 always contains the value 0. Unlike x86 and other complex instruction set architectures (ISAs), MIPS only operates on registers. This simplifies the ISA, but increases the number of instructions.
For example: `Mem[0x1001000] += Mem[0x1001004]` translates to:

- Read `Mem[0x1001000]` into register.
- Read `Mem[0x1001004]` into register.
- Compute the sum of both registers.
- Write the result back to `Mem[0x1001000]`.

Having simpler instructions means we can have fewer of them. With MIPS, we have the first 6 bits as the opcode, and the next 26 bits as the argument:

opcode	arguments
6 bits	**26 bits**

```
0x3c011001
0b00111100000000010001000000000001
0b 001111 00000000010001000000000001
     opcode          arguments
```

1.6 Instruction Types

In MIPS32, both addresses and instructions are 32-bit long. For instructions, 6 out of the 32 bits are used for opcodes. Therefore, we need to be able to encode addresses to be able to jump locations or load array addresses into pointers, and set 32-bit values into registers. We have three instruction types:

1. **R**-ithmetic (between register) operations, such as `add $8, $1, $2` and `sub $12, $6, $3`;
2. **I**-mmediate (including fixed numbers) operations, like `addi $15, $15, -1` and `beq $10, $14, 0x00000002`;
3. **J**-ump operations including `j 0x004000050` and `jal 0x00400007c`

The MIPS Reference Card is on the course Moodle website.

1.7 R-ithmetic Instructions

opcode	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

`R[rd] = func(R[rs], R[rt])`

`add $8, $1, $2` `0x00224020`

000000 00001 00010 01000 00000 100000

opcode rs = \$1 rt = \$2 rd = \$8 shamt funct = 20_{hex}

`sub $5, $7, $6` `0x00e62822` R[rd] = R[rs] - R[rt]

$$\underbrace{000000}_{\text{opcode}}\ \underbrace{00111}_{\text{rs}=\$7}\ \underbrace{00110}_{\text{rt}=\$6}\ \underbrace{00101}_{\text{rd}=\$5}\ \underbrace{00000}_{\text{shamt}}\ \underbrace{100010}_{\text{funct}=22_{\text{hex}}}$$

`sltu $3, $22, $11` `0x02cb182b` R[rd] = (R[rs] < R[rt]) ? 1 : 0

$$\underbrace{000000}_{\text{opcode}}\ \underbrace{10110}_{\text{rs}=\$22}\ \underbrace{01011}_{\text{rt}=\$11}\ \underbrace{00011}_{\text{rd}=\$3}\ \underbrace{00000}_{\text{shamt}}\ \underbrace{101011}_{\text{funct}=2b_{\text{hex}}}$$

1.7.1 Shift Instructions

`srl $11, $7, 4` `0x00075842` R[rd] = R[rt] $\gg$ shamt (rs is 0)

$$\underbrace{000000}_{\text{opcode}}\ \underbrace{00000}_{\text{rs}=0}\ \underbrace{00111}_{\text{rt}=\$7}\ \underbrace{01011}_{\text{rd}=\$11}\ \underbrace{00100}_{\text{shamt}=4}\ \underbrace{000010}_{\text{funct}=02_{\text{hex}}}$$

$100_{10} \ll 4 = 100 \cdot 2^4 = 1600$ $\quad$ $1100100_2 \ll 4 = 1100100\underline{0000}_2$

$100_{10} \gg 4 = \left\lfloor \frac{100}{2^4} \right\rfloor = 6$ $\quad$ $1100100_2 \gg 4 = 110_2$

$-100_{10} \gg 4 = ?$ It depends on the kind of shift:

Logical: `0xffffff9c >>> 4` = `0x0ffffff9` = 268435449_{10}

Arithmetic: `0xffffff9c >> 4` = `0xfffffff9` = $\left\lfloor \frac{-100}{2^4} \right\rfloor = -7$

1.7.2 Pseudo Instructions

There also exists *pseudo* instructions that are not real MIPS32 instructions (i.e. they do not have opcodes and aren't recognised by the CPU), but they can be translated into real instructions by the assembler. For example, `move $8, $9` can be translated into `add $8, $9, $0`.

1.7.3 High-level Commands

Going one way, we have the **disassembly** of machine code to assembly code and **reverse engineering** of assembly code to high level code like `x = a - b + c - d;`. In contrast, going from a high level language to assembly is **not unique**. There are multiple heuristics for optimisation, but going from assembly to machine code **is** unique.

1.7.4 Example of R-Type Instructions

Instruction	op	funct (hex)	Description
`add`	0	0x20	Addition, with signed overflow
`sub`	0	0x22	Subtraction, with signed overflow
`and`	0	0x24	Logical And
`or`	0	0x25	Logical Or
`xor`	0	0x26	Logical Xor
`nor`	0	0x27	Logical Nor
`slt`	0	0x2A	Set on less than
`sltu`	0	0x2B	Set on less than unsigned
`srl`	0	0x02	Shift Right Logical
`sra`	0	0x03	Shift Right Arith.
`sll`	0	0x00	Shift Left Logical

1.8 I-mmediate Instructions

Memory addresses are 32-bit long, as are the registers. A single way to compute a memory address:

$$\text{Base (register value)} + \text{Displacement (Imm value)}$$

$$\underbrace{\texttt{0x20}}_{\text{Displacement}} \quad (\underbrace{\texttt{\$8}}_{\text{Base}})$$

If `$8 = 0x10010000`, then `0x20($8)` is the address `0x10010020`. Working with integer arrays:

- `A[8] = y;`
- Say `$8` stores the address of `A` and `$9` stores y

Store Word (`sw`, I-type, opcode `0x2b`): $M[R[rs] + \text{SignExtImm}] = R[rt]$

$$\texttt{sw \$9, 0x20(\$8)} \implies \underbrace{\texttt{101011}}_{\text{op}}\ \underbrace{\texttt{01000}}_{rs=\$8}\ \underbrace{\texttt{01001}}_{rt=\$9}\ \underbrace{\texttt{0000000000100000}}_{\text{imm=0x20}} = \texttt{0xad090020}$$

Load Word (`lw`, I-type, opcode `0x23`): $R[rt] = M[R[rs] + \text{SignExtImm}]$
Example: `z = A[10];` loads `A[10]` into `$10`.

$$\texttt{lw \$10, 0x28(\$8)} \implies \underbrace{\texttt{100011}}_{\text{op}}\ \underbrace{\texttt{01000}}_{rs=\$8}\ \underbrace{\texttt{01010}}_{rt=\$10}\ \underbrace{\texttt{0000000000101000}}_{\text{imm=0x28}} = \texttt{0x8d0a0028}$$

Note: the address $R[rs] + \text{SignExtImm}$ must be word-aligned.

Immediate memory instructions also works with byte and half word granularities, and supports zero-extensions.

- Load/store bytes (`lb` / `sb`)
- Load/store half-word (`lh` / `sh`) (half-word aligned)
- Are sign-extended

1.8.1 I-mmediate value assignment

In Java/C, we may want to do a variable assignment such as `int x = 7654321;`. How would that be translated to MIPS?

opcode	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

The pseudo-instruction `li $8, 7654321` is expanded by the assembler into two CPU instructions, since $7654321_{10} =$ `0x74cbb1` does not fit in a 16-bit immediate:

$$\underbrace{\texttt{li \$8, 7654321}}_{\text{pseudo-instruction}} \implies \underbrace{\begin{array}{l}\texttt{lui \$1, 0x0074}\\ \texttt{ori \$8, \$1, 0xcbb1}\end{array}}_{\text{CPU instructions}}$$

- **Load upper immediate** (`lui`) sets the most significant 16 bits to the immediate and clears the lower 16:

  $$\texttt{\$1} = \texttt{0x00740000}$$

- **Or immediate** (`ori`) performs a bitwise OR with the zero-extended immediate `0xcbb1`:

  $$\texttt{\$8} = \texttt{0x00740000} \mid \texttt{0x0000cbb1} = \texttt{0x0074cbb1}$$

1.8.2 I-mmediate branching instructions

Essential for implementing loops. There are many branches on comparisons with zero: `bgez`, `bgtz`, `blez`, `bltz`, etc.